

diffusion-tinker: verification and reproduction

Srijit Iyer

June 2026

diffusion-tinker is a TRL-style library for RL post-training of diffusion models. You give it a model, a reward, and a list of prompts, and it runs one of six algorithms (FlowGRPO, DDRL, DRaFT, DiffusionDPO, DDPO, SFT) behind a single `Trainer/Config` API, on top of HuggingFace `diffusers`. It targets Stable Diffusion 3.5 and FLUX.1.

Before putting the library forward for a release, I wanted to be sure the algorithms were actually correct, not just that they ran without errors. This note is what came out of that: the bugs the verification caught, and the reproduction numbers on the fixed code. Everything below is a real run on a single NVIDIA A5000 (24 GB), except FLUX, which needed a 48 GB A6000-Ada.

1 What the verification caught

The library had a passing test suite and clean smoke runs, so on paper it looked fine. It was not. Checking that each trainer’s gradients actually reach the LoRA weights, and that the images leaving the VAE are in the range the rewards expect, turned up several real bugs. The two that mattered most:

DRaFT never trained. DRaFT backpropagates the reward through the last few denoising steps. Its sampling loop runs a no-grad warmup over the early steps and then a gradient-tracked loop over the last K , both inside one `torch.autocast` block. With autocast’s weight cache on (the default), the float32 LoRA weights get cast to bf16 and cached *during the no-grad warmup* as detached tensors; the gradient loop then reuses those detached casts, so no gradient ever reaches the adapter. `loss.backward()` left every LoRA `.grad` at `None` and the optimizer was a no-op. Disabling the cache fixes it. Before: reward flat at 5.4 for 30 epochs. After: 5.9 to 9.6.

Every generated image was about half black. The SD3 and FLUX VAEs decode to pixels in $[-1, 1]$, but the code clamped the decoder output straight to $[0, 1]$ without the usual $x/2 + 0.5$ rescale. A quick runtime check put 55% of the pixels in every image below zero, all crushed to black. The symmetric bug was on the encode side: `ToTensor` images in $[0, 1]$ were handed to a VAE that expects $[-1, 1]$, so SFT, DDRL, and DiffusionDPO were all training toward mis-scaled targets. Both are one-line fixes, and they touch essentially every image the library produces.

The rest were smaller but real: a sampling/replay precision mismatch that biased the PPO importance ratio to 0.97 instead of 1.0; three trainers calling `disable_adapter_layers()`, a method that does not exist on `diffusers` transformers, which crashed their reference-model paths; a DRaFT gradient-accumulation normalizer off by one; a missing stop-gradient in the GRPO ratio guard; and a stat-tracker that leaked memory across epochs.

The common thread is that none of these show up in a test that only checks the code runs. A smoke test that generates an image and computes a reward passes happily while the image is half black and the model is not learning. The fix was to verify the things underneath: gradients reaching weights, pixel ranges, ratio calibration.

2 Reproduction

All numbers are on the fixed code. Because the earlier “validated” results were measured on broken images and latents, every algorithm was re-run.

Algorithm	Task / reward	Result	
FlowGRPO	OCR text rendering	0.64 → 1.00 best eval accuracy	validated
DRaFT	Aesthetic (LAION)	5.92 → 9.58 mean reward	validated
DiffusionDPO	Preference (real vs. noise)	0 → 60 implicit reward	validated
DDRL	Aesthetic + data anchor	3.77 → 3.89, stable	validated
SFT	Naruto denoising	0.189 → 0.185 eval loss	validated
DDPO/DPOK	Aesthetic	ratio fixed to ~ 1.0 , reward flat	mechanism only

A few of these deserve a word. FlowGRPO with the OCR reward is the headline: it reaches a perfect 1.00 eval accuracy by epoch 10 (the paper baselines are 0.823 for DDRL, 0.845 for FlowGRPO), beating the library’s old 0.95 figure simply because the images are now exposed correctly and the OCR model can read the rendered text. The eval is coarse (four prompts, each scored 0 or 1) and noisy, so it oscillates between 0.75 and 1.00 after that; `save_best` keeps the perfect checkpoint. DRaFT is the cleanest curve, a smooth climb from 5.9 to 9.6. DiffusionDPO learns the preference within twenty steps: the winner’s denoising loss drops relative to the reference while the loser’s rises, exactly as the objective intends.

3 Where it is honest about its limits

DDPO is the one algorithm I will not claim a reward curve for. The precision fix corrected its importance ratio to ~ 1.0 , the gradients flow, and it shares FlowGRPO’s verified replay code, so the mechanism is right. But PPO is sample-hungry, and at the sample budget a single 24 GB card allows, the reward does not visibly climb. Showing a gain would need a much larger batch than fits here. It is included for completeness as the original diffusion RL method, with that caveat stated plainly.

FLUX runs. On FLUX.1-schnell the entire code path checks out at runtime: model auto-detection, packed latents, the sampling and replay log-probs, the schnell guidance handling, and the image decode. The eval aesthetic reward improves from 6.05 to a best of 6.39, but noisily. Two things hold it back: schnell is a four-step *distilled* model and was never meant for the stochastic SDE sampling these RL methods rely on, and its importance ratio sits around 0.92 rather than 1.0, leaving a small residual bias. FLUX.1-dev, which is not distilled, would be a better target and is the obvious next run.

4 Summary

Five of the six algorithms reproduce on Stable Diffusion 3.5, with the OCR result above the paper baselines, and the FLUX code path is verified on its native model. More to the point, the implementations are now checked at the level that actually matters for correctness rather than at the level of “it ran.” The fixes and the per-run numbers are in the repository at <https://github.com/srijitayer/diffusion-tinker>.